A Contract-Gated Imitation Learning Stack for Autonomous Driving in CARLA, and the Rendering Failures That Gate Its Data Collection

DRAFT v0.3 — INCOMPLETE MANUSCRIPT — NOT FOR CIRCULATION

Medhansh Khattar (25BAI10672), Prashant Kumar Jaglan (25BAI10750), Yash Yadav (25BAI10545)
Samyak Jain (25BAI10746), Sanchit Kumar Sinha (25BAI10229), Aryan Garg (25BAI10215)
School of Computer Science and Engineering
Vellore Institute of Technology, Bhopal, India

Abstract*—Learning a driving policy from demonstration is usually reported as a modelling result. In our experience the binding constraint has been elsewhere: we cannot reliably render the environment. This paper describes an imitation-learning stack built on CARLA 0.9.16 around a deliberately small state-based policy (an $8 \to 24 \to 16 \to 3$ perceptron, 667 parameters) and three explicit contracts: an eight-field observation contract, a seven-field action protocol, and a deployability contract an artifact must satisfy before it may drive. An independent safety supervisor, outside the learned policy, retains full braking authority. We report the demonstration protocol and the route-level split that avoids frame-level leakage; we then report, at length, the rendering failures that dominate the project. The final camera image cannot be rendered at speed on an integrated GPU, environment streaming stalls the simulator, and the resulting crashes destroy episodes mid-route. Because our acceptance rule discards any interrupted route, these crashes propagate directly into an empty training corpus. *Quantitative evaluation is therefore unavailable: the accepted corpus stands at zero routes of a twelve-route target, and no artifact satisfies the v2 contract. Results are reserved for Sections VII–IX and remain unwritten.

***Index Terms*—autonomous driving, imitation learning, behaviour cloning, CARLA, rendering bottleneck, safety supervision, reproducibility.**

I. Introduction

Simulation is the default proving ground for learned driving policies, and CARLA [1] is the most widely adopted open platform: it exposes an Unreal Engine world over an RPC interface, supplies OpenDRIVE road topology, traffic and weather, and permits deterministic replay of a seeded route. Failures cost nothing — provided the simulation runs at all.

Much of the literature following Bojarski *et al.* [2] maps camera pixels directly to control. We take a narrower first step: our policy consumes eight route-relative scalars rather than images. The motivation is the size of the demonstration corpus a six-person undergraduate team can realistically curate — with a few thousand frames a larger network memorises the road geometry of the training town instead of learning to hold a lane [4]. Our contribution is an engineering one: the architecture, the contracts holding it together, and — centrally — the rendering and stability failures met in building it. Where a driving result would ordinarily appear, this draft states plainly that it does not yet exist, and Section VI explains why.

II. Related Work

End-to-end behaviour cloning for driving was demonstrated at scale by [2]. Codevilla *et al.* [3] added a high-level command input for intersection ambiguity, and later documented behaviour cloning's own limitations [4]: distributional shift and bias toward the modal action. Ross *et al.* [5] give the standard remedy, DAgger. Within CARLA, Learning by Cheating [6] shows privileged-to-sensorimotor distillation outperforming direct cloning, and TransFuser [7] sets the multi-modal baseline; our observation contract is closer to the privileged stage of [6].

> *TODO — RELATED WORK. Not yet cited, still to be read and added: (i) Toromanoff et al., implicit affordances (CVPR 2020), and where our reward logging sits relative to it; (ii) Kiran et al., DRL survey (T-ITS 2021), for positioning; (iii) SAE J3016, so the autonomy claim is precise; (iv) prior work on simulator throughput and headless rendering — unsurveyed, and directly relevant to Section VI. Owner: unassigned.*

III. System Architecture

The stack spans two machines on a direct Ethernet link. Machine 1 runs the CARLA server and owns the world: physics, the OpenDRIVE map, traffic manager, sensors and the vehicle. Machine 2 runs policy inference, the route planner, an observer window and network visualisation. Each iteration: machine 1 publishes world state; machine 2 runs the planner and policy; machine 2 returns a driving command; machine 1's safety layer validates it and only then applies it.

The map is deliberately split: CARLA's physical map stays on machine 1 because the simulator owns the world, while the navigation map that picks a destination and computes waypoints may live on machine 2, needing only road topology. A watchdog on machine 1 should brake fully if the link drops; it is implemented but untested under induced failure.

IV. Methodology

A. Observation contract

One module, `policy_contract.py`, authoritatively defines what the network sees: eight scalars, each clamped by a shared normaliser (Table I).

TABLE I
THE EIGHT-FIELD OBSERVATION CONTRACT

Field	Meaning	Field	Meaning
`lane_error_m`	offset from lane centre	`front_obst_m`	nearest car ahead
`heading_err_deg`	car vs. road angle	`red_light`	binary: light governs
`speed_kmh`	current road speed	`curvature`	bend ≈8 m ahead
`target_spd_kmh`	speed controller wants	`stopped`	binary: <1 km/h

Route-relative rather than world coordinates is what allows a policy trained on one town to be evaluated on another. One gap is open: the route-reference waypoint used for heading and curvature at training time is not yet supplied identically at inference, so a train/serve skew remains possible on two fields. Any result reported before it is closed would be unsound.

B. Action protocol and safety supervision

The policy emits three values; the vehicle interface accepts seven. Steering lies in $[-1, +1]$ and maps to $\pm 70°$; accelerator and brake are clamped to $[0, 1]$, and the runtime refuses to apply a meaningful accelerator and brake simultaneously. The final layer is linear — all clamping is done by the runtime, not the network. The remaining four fields (clutch, handbrake, wipers, gear) come from a deterministic six-speed transmission model, and engine speed, which CARLA does not simulate, is derived from road speed and gear ratio and labelled *estimated telemetry* wherever displayed.

A safety supervisor runs on the simulator host, outside the policy, and forces a full brake on any of: collision, sensor timeout, a command older than 0.5 s, lane error above 2 m, an obstacle within 7 m, speed above 50 km/h, a red light, or invalid telemetry. It is the only component permitted to override the network.

C. Demonstration collection

Two experts are used: CARLA's rule-based `BasicAgent`, and a human at a six-zone cockpit. A route runs from spawn to a destination at least 150 m away, and every tick writes one JSONL row holding the eight observations, the expert's three controls, telemetry and metadata. A route is promoted only if it is collision-free, completed, and holds $\geq$100 samples; crashes and interrupted runs are discarded rather than truncated. **This rule is the mechanism by which the rendering failures of Section VI convert into missing data.**

All accepted recordings use Town05, with Town04 held out as an unseen map so a good score cannot be explained by memorised road geometry. Twelve Town04 teacher episodes recorded before this rule was adopted were quarantined rather than deleted.

D. Training pipeline

Training is offline, in a Colab notebook, and produces a JSON artifact the local driver loads directly. The model is a linear $8 \to 24$ layer, $\tanh$, $24 \to 16$, $\tanh$, and a linear $16 \to 3$ head: 667 parameters. Optimisation uses AdamW ($\mathrm{lr} = 10^{-3}$, weight decay 10^{-5}) against mean squared error on the three control channels, batch size 128, at most 80 epochs, early stopping after 12 without improvement. A single seed (7) is set across Python, NumPy and PyTorch; training runs on CPU.

The split is taken at route level, 70/15/15, never at frame level. Consecutive frames within a route are near-identical, so a random frame split places duplicates in both training and test sets and renders the reported error meaningless. Each episode goes to exactly one split, and an assertion confirms the sets are disjoint. The notebook generates its normaliser verbatim from the contract file, so training-time normalisation cannot diverge from inference-time.

E. Deployability gate

An exported artifact carries a policy-contract version, a `deployable` flag and the episode names in each split. The live driver refuses any artifact whose contract version is not v2 or whose flag is false. At present only a synthetic smoke-test artifact exists (contract v1, `deployable: false`), so it is refused and route mode is unavailable — and there is no scripted-agent fallback, so the car is simply not driven autonomously. Training is hard-disabled at runtime, so the deployed system cannot modify its weights.

One runtime behaviour must be declared, since it is not learned: cloning over-represents braking, so the policy stalls on a clear road. A bounded recovery capped at 0.35 accelerator releases it, disabled instantly by a red light, an obstacle or invalid telemetry, and never edits the weights. Any result must state whether it was active.

V. Experimental Setup (partial)

The scenario catalogue defines four categories so a policy can be stressed across road type and traffic density without hard-coding a map: urban (Town03/04/05/10HD, 30 NPCs), highway (Town05/06/07, 20), rural (Town01/02, 12) and intersection (Town03/04, 40), with eight weather presets and three pothole modes. These are *configured* definitions only — **no category has yet completed a verified evaluation run**, for the reasons in Section VI.

> *TODO — SECTION V. Missing: a hardware table for both machines; control-loop latency over the link (mean, p99); headless tick rate; the commit hash for the reported runs.*

VI. The Rendering Bottleneck

This section is the honest centre of the paper. The dominant obstacle has not been the learning algorithm; it has been getting the environment and the final camera image to render at all, for long enough to complete a single route.

A. The final image will not render at speed

The visual cockpit composites four camera feeds alongside streamed scenery. On an Intel Arc 140V integrated GPU this saturates the device: the frame rate collapses as road speed rises, because faster travel forces Unreal to stream new scenery while the same device is already compositing the camera panel. The failure is not graceful: the image stalls, the client falls behind the server tick, commands go stale.

Two mitigations are in place, both costing realism. The default neural path runs *headless*, with no camera feeds rendered at all. On the visual build, throttle is removed above 42 km/h and top speed capped at 48 km/h purely to protect the GPU. The consequence must be stated: the speed range for visual demonstrations is bounded by a rendering limit rather than a driving decision, so no highway-speed behaviour can currently be demonstrated.

B. Environment rendering stalls crash the stack

Environment rendering fails in three ways, all ending the episode:

- **Thermal termination.** The integrated GPU shares a thermal package with the CPU. Sustained rendering drove a fatal hardware error and a critical-thermal hibernation in one afternoon. No live temperature counter is exposed here, so a health guard reads the Windows Event Log instead and refuses to launch inside a 30- or 60-minute cooldown.
- **Silent process death.** A rendering component wedges, Windows kills it as a hung application, and the stack goes down. Nothing was logged originally, because the launcher fired everything into detached tasks whose output went nowhere. One supervisor now owns both processes, writes real logs, probes CARLA's RPC port for liveness rather than trusting a PID, and restarts with $5\,\text{s} \rightarrow 120\,\text{s}$ backoff.
- **Orphaned renderers.** `CarlaUE4.exe` is a shim around another executable, and the virtualenv `python.exe` re-executes the real interpreter as a child. Killing the visible process left the real renderer holding port 2000, so the next launch fought the corpse of the previous one. The supervisor now kills process trees and sweeps orphans at startup.

C. Why this empties the training corpus

These are not merely operational annoyances; they are why Section VII is blank. A rendering stall interrupts a route in progress, and by the rule of Section IV-C an interrupted route is discarded rather than kept — a deliberate choice, since a route ending in a crash teaches the policy that crashing is normal. The two rules compose badly: the more the environment fails to render, the fewer routes survive, and at present **zero routes have been accepted** of twelve. The model is not failing to learn; it has not been given anything to learn from.

> *TODO — SECTION VI. Qualitative; make quantitative before submission. Needed: (a) a figure of FPS vs. road speed, headless vs. visual; (b) mean route survival time before a rendering-induced termination; (c) counts per failure class; (d) package temperature at termination (unobtainable — state the limitation); (e) whether the two-machine split relieves the bottleneck.*

VII. Results

This section is not yet written.

The corpus does not exist, for the reasons in Section VI, and a training loss against the smoke-test artifact would be misleading. To prevent metric selection after the fact, the reporting format is fixed now: collisions per 100 km, lane departures per km, route completion rate, safety interventions per route, and per-channel MAE — each for the rule-based baseline and the learned policy on the held-out map, and every one of them *pending*.

> *TODO — SECTION VII. Prerequisites, in order: (1) stabilise rendering so a 150 m route survives end-to-end; (2) accept $\geq$12 Town05 routes; (3) close the train–serve skew of Section IV-A; (4) export a contract-v2 artifact with `deployable = true`; (5) run the Town04 evaluation with the stall assist on and off. No prose before (5) completes.*

VIII. Discussion

Stub. Only observations independent of quantitative results are recorded.

The project's time went to rendering and process management rather than modelling, worth reporting because it is absent from the work that motivated ours. Every result in [6], [7] presupposes a simulator that renders reliably; where that fails, the bottleneck moves somewhere the literature does not discuss.

> *TODO — SECTION VIII. Once VII exists: (a) a leakage demo — error under route- vs. frame-level splitting; (b) a failure taxonomy from Town04; (c) what eight scalars cannot represent (pedestrians, signs, any cue needing vision); (d) whether headless collection biases the policy relative to the visual build.*

IX. Conclusion and Future Work

[Not written. To be drafted after Sections VI and VII are complete.]

Placeholder for planned direction: resolving the rendering bottleneck, if necessary by relocating the renderer to dedicated hardware; migration from eight scalars to RGB, depth and semantic segmentation; conditional imitation learning [3]; DAgger-style correction [5]; and a curriculum across the catalogue. Reward logging already runs each tick, but no online loop consumes it.

References

[1] A. Dosovitskiy *et al.*, "CARLA: An Open Urban Driving Simulator," *CoRL*, 2017. arXiv:1711.03938.

[2] M. Bojarski *et al.*, "End to End Learning for Self-Driving Cars," NVIDIA, 2016. arXiv:1604.07316.

[3] F. Codevilla *et al.*, "End-to-end Driving via Conditional Imitation Learning," *ICRA*, 2018. arXiv:1710.02410.

[4] F. Codevilla *et al.*, "Exploring the Limitations of Behavior Cloning for Autonomous Driving," *ICCV*, 2019.

[5] S. Ross, G. Gordon, and J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," *AISTATS*, 2011. arXiv:1011.0686.

[6] D. Chen *et al.*, "Learning by Cheating," *CoRL*, 2019. arXiv:1912.12294.

[7] A. Prakash, K. Chitta, and A. Geiger, "Multi-Modal Fusion Transformer for End-to-End Autonomous Driving," *CVPR*, 2021.

TODO — REFERENCES incomplete: add CARLA docs, OpenDRIVE, rendering refs.